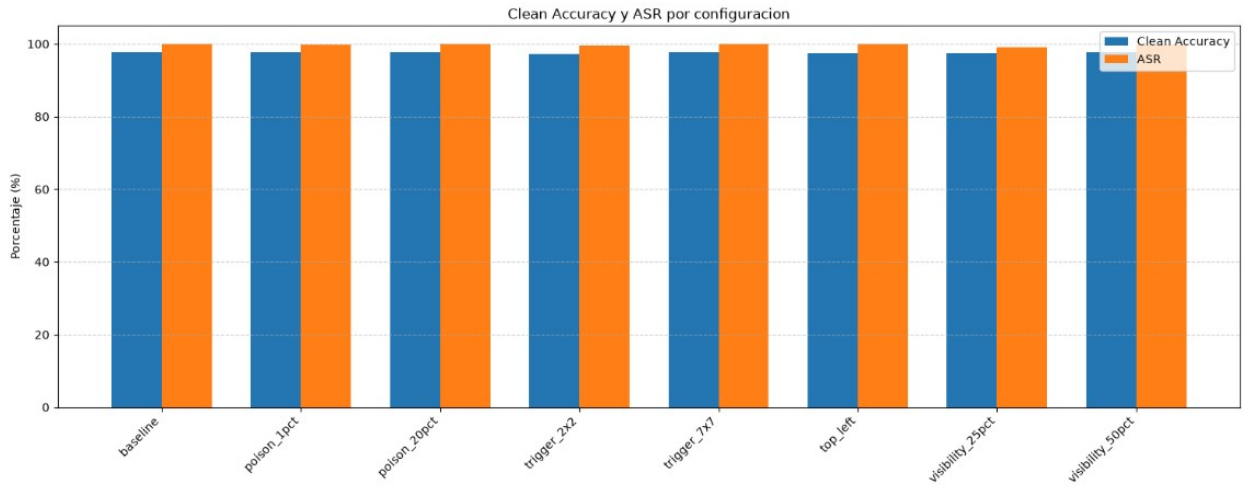
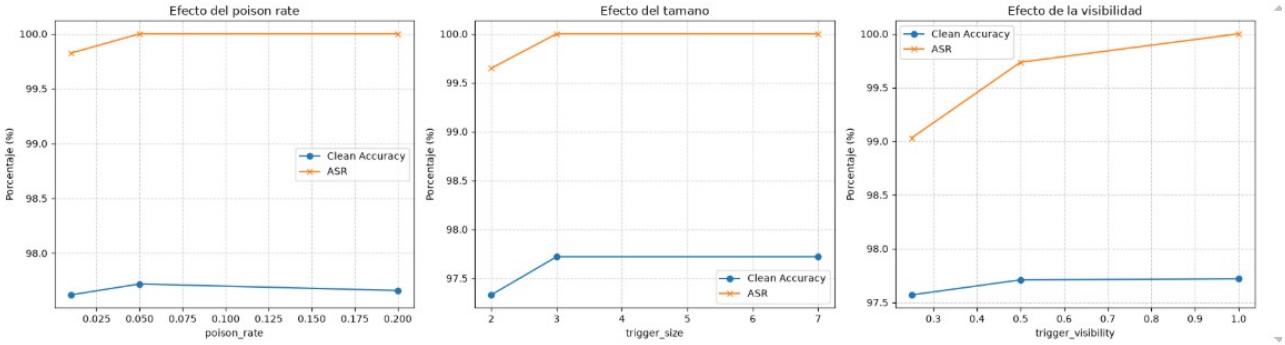
Parte II: Desarrollaremos, ejecutaremos y analizaremos técnicas de envenenamiento de datos (data poisoning), ataques de tipo backdoor y métodos para su detección y mitigación.

IIa) Analiza en detalle el código P3-Src1.ipynb para comprender el funcionamiento de un ataque de tipo BadNets [P3a]. Explica la lógica general del código, describiendo las distintas etapas del proceso, los datos utilizados, la construcción del trigger, el mecanismo de envenenamiento del conjunto de entrenamiento y la evaluación del modelo resultante.
El notebook comienza declarando ciertas variables importantes para este ataque, como POISON_RATE, SOURCE_CLASS; TARGET_CLASS, además de otras normales como batchsize, epochs, lr que se usan en el entrenamiento de modelos en ml de toda la vida.
Se define una función add_trigger que clona la imagen de entrada y añade en Height, Width desde el 25 hasta el 28 un cuadrado con “intensidad” de pixel al máximo. Osea en esquina inferior derecha supongo que las imágenes son 28x28. Después se crea la clase BadNetMNIST que hereda de Dataset que viene de torch utils data. Aquí se define el source class, target class, dataset, osea este será el dataset que coge el original y lo convierte en el malicioso. Tenemos dataset.targets que devuelve las etiqeutas reales de las imágenes de entrenamiento, un tensor enorme básicamente con los labels. Lo que se hace es declarar source_indices que mira los targets cuyo valor es el mismo a source_class y asi tenemos el número de elementos con sus posiciones que nos interesan que tiene ese source class declarada antes, osea las posiciones donde tenemos True. Esto es básicamente porque queremos envenenar imágenes que estan en el source class, y el numero se calcula con el numnero de sources * el rate declarado, y los idn ices posion donde se hará este cambio de etiqueta y se añadirá el trigger será escogido de forma random de esta lista creando un set con los indices seleccionados. En getitem se devuelve image, label pero la imagen se le añade el trigger y a label se le cambia según el target class declarado.
Después se crea un dataset de test que hace lo mismo de sacar los índices según el source_class, y en getitem deveuvel el indice real y añade el trigger, de esta forma se sabe dónde cambia la predicción del modelo respecto del gt. Básicamente el otro dataset era para entrenar el modelo, este es para probar si funciona cuando se añade el trigger. Se crea otro modelo una simple net que es una red fully connected.
Implementa y evalúa diferentes configuraciones experimentales modificando parámetros relevantes del ataque, tales como el poison rate, el tamaño, la posición o la visibilidad del trigger. Analiza cómo afectan estas modificaciones a la precisión sobre muestras limpias (Clean Accuracy) y a la tasa de éxito del ataque (ASR). Representa gráficamente los resultados obtenidos y extrae las conclusiones correspondientes, discutiendo el compromiso entre efectividad del ataque, degradación del rendimiento del modelo y detectabilidad del trigger.

	name	poison_rate	trigger_size	trigger_position	trigger_visibility	clean_accuracy	asr
0	baseline	0.05	3	bottom_right	1.00	97.72	100.00
1	poison_1pct	0.01	3	bottom_right	1.00	97.62	99.82
2	poison_20pct	0.20	3	bottom_right	1.00	97.66	100.00
3	trigger_2x2	0.05	2	bottom_right	1.00	97.33	99.65
4	trigger_7x7	0.05	7	bottom_right	1.00	97.72	100.00
5	top_left	0.05	3	top_left	1.00	97.52	100.00
6	visibility_25pct	0.05	3	bottom_right	0.25	97.57	99.03
7	visibility_50pct	0.05	3	bottom_right	0.50	97.71	99.74





El trigger de menor tamaño tieneun ASR un poco menor, pero sigue siendo efectivo. Los otros triggers consiguen mayor rendimiento , aunque el de 7x7 que es más grande es más facil de detectar visualmente. El de 3x3 en este caso es el más equilibrado. En un caso real donde haya que capturar, uno más grande igual es mejor opción porque es más fácil de capturar desde diferentes ángulos y más invariable a transformaciones que uno pequeño. Cuanto más visiblea el trigger, vemos que más ASR

IIb) Analiza en detalle el código P3-Src2.ipynb, correspondiente a un ataque de tipo Clean-Label Poisoning basado en el artículo Poison Frogs! [P3b]. Explica las diferencias fundamentales respecto al enfoque BadNets, prestando especial atención al proceso de generación de los ejemplos envenenados y a la ausencia de modificaciones explícitas en las etiquetas de entrenamiento. Evalúa experimentalmente la eficacia del ataque para diferentes cantidades de muestras envenenadas y extrae las conclusiones correspondientes.

Tras analizar el código, las principales diferencias con BadNets son las siguientes. En BadNets, las etiquetas se modifican para que el modelo aprenda a clasificar de forma errónea. En cambio, en este ataque de Clean-Label Poisoning, como su nombre indica, las etiquetas siguen siendo las correctas. Lo que se hace es escoger una clase base, en este caso frog, y una clase objetivo o target, que es bird. Se trata de crear muestras poison que visualmente, para un humano, parecen de la clase base (es decir, parecen una rana), pero que en el espacio de características del modelo, es decir, en su espacio latente, se representan como un bird

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \; \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

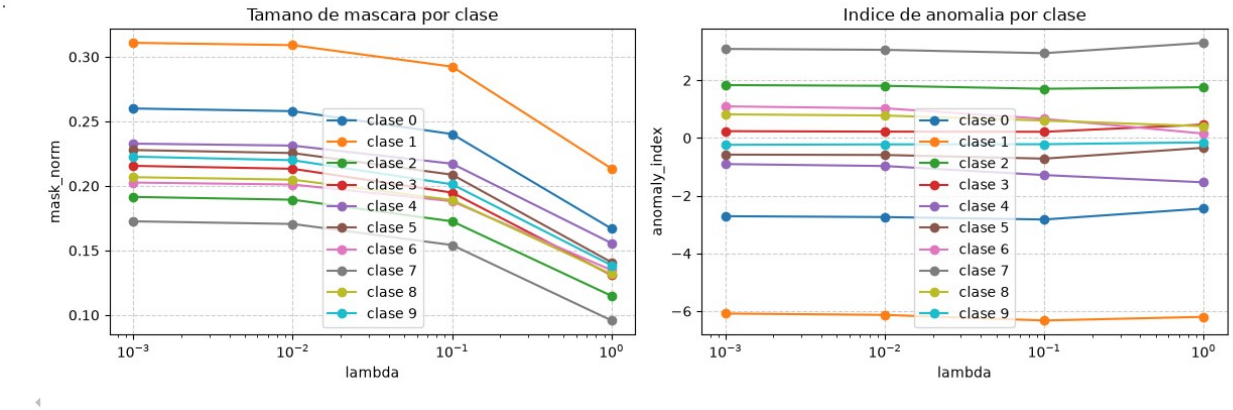
REPR. VISIÓN

En el código esto se ve cuando se llama al ataque FeatureCollisionAttack de la librería. Se crean así las muestras poison: imágenes que visualmente siguen pareciendo frog, pero que en el espacio latente del modelo son similares a bird. De esta forma, si hacemos una predicción con el modelo sin reentrenar, al pasarle una de estas imágenes poison, el modelo dirá bird, porque sus características internas son las de un bird, aunque para nosotros visualmente sea una rana.

Ahora bien, cuando reentrenamos con el conjunto posioned, la etiqueta de esa muestra poison ha sido revisada por un humano. Ese humano ve una rana, por lo que la etiqueta asignada es frog. Entonces el modelo termina aprendiendo que unas características internas propias de bird corresponden a la clase frog. Como la instancia target bird está cerca de la muestra poison en el espacio de características, el modelo también la arrastra hacia la región de frog. Por eso se escogió target_class = bird: ahora, dada una imagen de bird, el modelo dirá que es frog, porque ha aprendido que sus características (las de bird) tienen etiqueta frog, aunque visualmente las fotos sean de bird.

IIc) Aplica una técnica de detección basada en Neural Cleanse [P3c] sobre los modelos de ataque basados en triggers. Analiza los resultados obtenidos e interpreta si el método es capaz de detectar la presencia de una puerta trasera en diferentes escenarios experimentales.
Sabemos que Neural Cleanse intenta reconstruir para cada clase objetivo, la mascara y el patron minimo que hacen que muchas entradas sean clasificadas como esa clase. La puntuacion `mask_norm` mide el tamano de la mascara inferida. Se usa la desviacion absoluta mediana (MAD) porque es robusta frente a valores extremos; una clase con una mascara mucho menor que las demas se marca como anomalía. Se ha aplicado para diferentes tamñaos de máscara ajustando un parámetro de regularización:

	regularization	max_anomaly_class	mask_norm	reconstruction_success	anomaly_index
0	0.001	7	0.1726	100.0	3.0875
1	0.010	7	0.1705	100.0	3.0552
2	0.100	7	0.1542	100.0	2.9393
3	1.000	7	0.0958	100.0	3.2980



El trigger está diseñado para hacer que imágenes de clase 1 sean clasificadas como clase 7, y Neural Cleanse identifica la clase 7 como la clase asociada a la máscara anómala en todas las configuraciones probadas.

¿Cómo mitigamos los ataques de tipo Clean-Label?
La defensa debe de verificar la procedencia y tener control de acceso del dataset, tener deteccion de ejemplos atipicos mediante embeddings, vecinos cercanos y clustering, una auditoria de duplicados

y de muestras casi idénticas, implemetnar entrenamiento robusto, por ejemplo con augmentations fuertes, tener validacion separada y pruebas de backdoor, como Neural Cleanse. Y si se identifica un cluster o trigger sospechoso, se eliminan o aíslan esas muestras y se reentrena el modelo; no se debe cambiar automaticamente la etiqueta sin evidencia.

Opcional: ¿Puedes implementar una técnica de mitigación y evaluar experimentalmente su eficacia?